AI Dev & Vibe Coding

2026-04-19 · brisk pomegranate Haubentaucher

THE TOOLS CHANGED. THE FUNDAMENTALS DID NOT. KNOW BOTH.

Two Ways AI Helps You Code

AI CODING TOOLS come in two flavours, and knowing when to use which matters:

Terminal/CLI agents Claude Code (https://docs.anthropic.com/en/docs/claude-code), Aider (https://aider.chat/), Codex CLI work directly in your repo from the terminal, read your files, make changes, run commands. Best for: implementing features, debugging, refactoring across multiple files.

IDE-integrated agents Cursor (https://www.cursor.com/), Windsurf (https://windsurf.com/), Cline (https://cline.bot/) full IDE with AI built in, visual diffs, inline suggestions. Best for: editing in context, reviewing changes visually, inline completions while typing.

Autocomplete tools: GitHub Copilot (https://github.com/features/copilot). Chat-based tools: ChatGPT (https://chatgpt.com/), Claude (https://claude.ai/). CLI and IDE agents are the next level: they read your whole project, not just the current file.

The Agent Loop

How these tools actually work the agent loop repeats until the task is done:

- Observe: the agent collects context, your repo structure, git status, recent commits, the files you point it at.
- Inspect: it analyses the code and your request to understand what needs to change.
- Choose: it selects an action, edit a file, run a command, ask you a question.
- Act: it executes the action, gets the result, and loops back to observe.

The harness (the program wrapping the LLM) validates every action before execution: is the tool recognised? Are the arguments valid? Is the file path inside the repo? The LLM proposes, the harness gatekeeps.

The quality depends entirely on what context you give the agent. Garbage in, garbage out. A vague prompt produces vague code. A precise prompt with file references produces precise changes.

Sandboxing

AI agents run on your machine, so what stops them from doing damage? An unrestricted agent can run `rm -rf /`, `curl` your secrets to an external server, install packages, or modify system files. It runs with your user's permissions.

How sandboxing works in practice:

Permission prompts Agents like Claude Code ask before executing each command, you approve or deny. The human stays in the loop.

Code-only mode Aider is code-only by default, it reads and edits files but does not run shell commands at all.

Restricted execution Docker containers, VMs, or chroot jails, the agent runs in an isolated environment where it physically cannot access the host system.

Allow/deny lists Configure which commands or directories the agent can access. Some tools let you whitelist specific tools (e.g., "can run `pytest` but nothing else").

The principle: give the agent the minimum access it needs. Read your code, yes. Edit files, yes. Run arbitrary commands, only with explicit approval. Access `.env` or SSH keys, never.

Exercise: Set up Aider

Exercises are for practice and reinforcing concepts. If your VM is broken, restore the `B9-complete` snapshot. No snapshot needed after this session, this is the final block.

Set up Aider on your dev VM. Aider (https://aider.chat/) is an open-source CLI agent. Connect it to Qwen (https://qwen.readthedocs.io/) via Ollama (https://ollama.com/), self-hosted on the server, provided by the course. No external API needed:

```
pip install aider-chat
aider --model ollama/qwen2.5-coder
```

Once Aider runs, you have a working instance of the agent loop sitting at your prompt. The next exercises will use the same session as their starting point.

Git as a Safety Net

COMMIT BEFORE YOU LET AN AGENT TOUCH YOUR CODE. This is not just an AI workflow, it is how you work with any risky change: experimental refactors, dependency upgrades, merge conflict resolution.

```
# Save current state
git add -A && git commit -m "pre-agent checkpoint"

# See what the agent changed
git diff

# Revert everything if the agent made a mess
git checkout .

# Stash your current state before experimenting
git stash

# Let the agent work on a branch
git checkout -b experiment
# ... agent does its work ...
# If it works, merge. If not, delete the branch.
git checkout main
git branch -D experiment
```

Always commit clean state first, then change, then review the diff. This pattern predates AI tools, it is how experienced developers have always worked with risky changes.

Reading Diffs

THE CORE REVIEW SKILL. When an agent (or a colleague) makes changes, you do not re-read the entire file. You read the diff: what was added, what was removed, what was modified.

```
git diff                    # in the terminal
```

`git diff` IN THE TERMINAL, side-by-side diffs in the IDE, GitHub's pull request diff view, all the same skill. Students need to read diffs fluently. This is how code review actually works.

Lines starting with + were added. Lines starting with - were removed. Lines starting with a space are unchanged context. The `@@` header tells you which file and which line numbers changed.

Exercise: Commit Clean State

First rule, commit clean state before letting the agent touch anything:

```
git add -A
git commit -m "pre-agent checkpoint"
```

This is your undo button for everything that follows. From this clean state, every change the agent makes shows up cleanly in `git diff` and can be reverted with a single command.

Verify the safety net works. Let the agent make a small, deliberately throwaway change, then read the diff and revert:

```
git diff
git checkout .
```

You should land back at the pre-agent checkpoint with the agent's edits gone. Until you can do this without thinking, do not give the agent anything important to touch.

Context Management

Agents have a limited context window, how much text the LLM can process at once. Long conversations degrade quality: the agent forgets earlier instructions, repeats mistakes, or contradicts itself. Older parts of the conversation get compressed or dropped.

Practical rules:

- Start a fresh conversation when switching tasks.
- Point the agent at specific files rather than letting it read everything.
- Keep instructions files short and focused.

Subagents

Some tools can spawn child agents for subtasks, e.g., "research this question in a separate context, report back." The child agent operates with tighter permissions and its own context window, keeping the parent conversation clean.

Exercise: Explore the Tool

GET COMFORTABLE WITH THE CONVERSATION LOOP by asking the agent to explain a file, describe the PixelWise architecture, or find where a function is called. Try:

- "Explain what `app/classifier.py` does"
- "Where is `classify_batch` called?"
- "Describe the full request flow from browser to database"

WATCH THE CONTEXT FILL UP. Notice how the agent's answers degrade if you keep piling questions into the same conversation, and how a fresh session with a single, file-scoped prompt produces a sharper answer. Pointing the agent at specific files rather than asking it to "look at the whole project" is the cheapest quality gain you have.

Teaching the AI Your Project

INSTRUCTIONS AS CODE, declarative files that persist across sessions. You write them once, the AI follows them every time. This is the highest-leverage thing you can do with AI tools.

`CLAUDE.md` Project-level instructions for Claude Code: conventions, architecture, what NOT to do.

`.github/copilot-instructions.md` Same concept for GitHub Copilot.

`.cursorrules` Cursor-specific project rules.

THE PATTERN ACROSS ALL TOOLS: each has its own filename, but the idea is identical, a markdown file in your repo that the AI reads before starting work.

Skill Files

SKILL FILES ARE REUSABLE RECIPES for specific tasks, structured `.md` files with step-by-step instructions the AI follows consistently. Example: "how to add a new API endpoint to this project." The AI produces correct code when given only the skill file and a task description.

Exercise: Write a CLAUDE.md

Write a `CLAUDE.md` (or equivalent instructions file) for the PixelWise project. Document: the stack, the conventions, what the AI should and should not do. Example structure:

```
# PixelWise

## Stack
- Python 3.11, FastAPI, SQLAlchemy, PostgreSQL
- Nginx reverse proxy, vanilla JS frontend
- sklearn LogisticRegression model (MNIST)

## Conventions
- All config via .env (never hardcode secrets)
- Batch-first inference (classify_batch is the core)
- Pydantic models for all API contracts

## Do NOT
- Modify .env or expose secrets
- Add new dependencies without updating requirements.txt
- Change the classifier interface without updating tests
```

Commit it to the repo. From now on, every agent session in this project starts with this file already in context.

Exercise: Write a Skill File

Create a `.skills/` directory and write a skill for a specific task, e.g., "how to add a new API endpoint":

```
mkdir -p .skills
nano .skills/add-endpoint.md
```

Include: which files to modify, what patterns to follow, what to test. Give the skill file to the agent with a task description, does it produce correct code?

Iterate on the skill until the agent can carry out the task from the skill alone, without you correcting it mid-way. A good skill file is one a tired colleague could also follow.

Working with AI Agents Effectively

The agent-edit-verify loop:

- Agent makes changes.
- `git diff`, review every line.
- Run the app, works or does not.
- If not, feed the error back to the agent or fix it yourself.
- Iterate until it is right, or revert and try a different approach.

Use `/plan` to let the agent think before it acts, review the plan, adjust, then let it execute. Do not jump straight to implementation.

Code review with AI: have the agent review your own code, it catches things you are blind to after staring at the same file for hours.

Exercise: Make a Real Change with the AI

STUDENT'S CHOICE, a new feature, a refactor, a bug fix. Follow the agent-edit-verify loop:

- Let the agent make changes.
- Review: `git diff`, read every line.
- Run the app, does it work?
- If not, feed the error back. Iterate or revert.

STAY IN THE LOOP the entire time. The moment you stop reading the diffs is the moment the agent starts shipping bugs into your repo with your name on the commit.

Exercise: Branching Workflow

Create a branch for the AI's work so the main line stays clean while you experiment:

```
git checkout -b ai-experiment
# ... let the agent work ...
git diff main
# If good: merge. If bad: delete the branch.
```

The branch is the boundary between "the agent has free rein" and "the codebase you trust". Merge only what passes the diff and the run.

Exercise: Group Discussion

COMPARE NOTES WITH THE ROOM. When was the AI faster than writing code by hand? When was it slower? When was it confidently wrong? What did the instructions file change about the AI's output quality?

NAME ONE THING you will keep doing, one thing you will stop doing, and one habit you want to acquire before the agent touches production code on your behalf.

Optional: MCP and Tool Use

AGENTS CAN DO MORE THAN EDIT CODE. MCP (Model Context Protocol) lets agents connect to external tools, reading documentation, searching the web, running tests, querying databases, interacting with APIs. The agent is not limited to what is in your repo.

MCP: https://modelcontextprotocol.io/

Optional: When to Trust AI Output

Strong Boilerplate, tests for known behaviour, documentation, refactoring, explaining unfamiliar code.

Weak Security-sensitive code, novel algorithms, anything touching infrastructure or secrets.

Never Blind trust without reading the output.

Security Thread

AI-generated code must pass the same security review as human-written code. Prompt injection: user-submitted text that ends up in an AI prompt is an injection vector, never pipe untrusted input into an AI call. The AI must never be given access to `.env`, SSH keys, or production credentials directly.

A note on what comes next. This is the final block. The course wrap-up sits in the recap below: every layer you touched, from two empty VMs to a deployed, monitored, AI-assisted service, is now yours to maintain.

Self-Reflection and Recap

Reflection questions:

- Why do you commit clean state before letting an agent touch your code?
- What is the difference between a CLI agent and an IDE agent? When would you use each?
- How does an instructions file improve the quality of AI-generated code?
- What are the risks of giving an AI agent unrestricted access to your machine?
- When should you trust AI output, and when should you be skeptical?

Recap of key concepts:

- The agent loop (observe, inspect, choose, act) and the harness that gatekeeps every action.
- Git as the safety net: clean checkpoint, branch, diff, revert.
- Sandboxing and least privilege: the agent gets the minimum access it needs.
- Context management: short conversations, file-scoped prompts, instructions files.
- `CLAUDE.md` and skill files as the highest-leverage way to steer AI output.

Milestone. Students have used a terminal AI agent on a real codebase. Instructions file and skill file committed to the repo. Every student can read a diff, revert a bad change, and iterate with an agent.

Course wrap-up. Look at what you have built, from two empty VMs to a